

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

NAME: A. Suraj varma

Reg.No: 192525339

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Dr

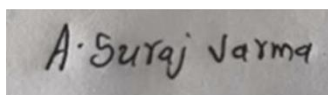
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5

7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

I A. Suraj varma (192525399) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1: Relational Algebra Operations

Relations

Employee(EmpID, EmpName, Salary, DeptID)

Department(DeptID, DeptName)

1. Selection (σ)

Retrieve employees whose salary is greater than 50,000.

$$\sigma_{Salary > 50000}(Employee)$$

2. Projection (π)

Display only Employee ID and Employee Name.

$$\pi_{EmpID, EmpName}(Employee)$$

3. Union ($\cup$)

Combine employees from Employee_A and Employee_B.

$$Employee_A \cup Employee_B$$

4. Set Difference ($-$)

Find employees in Employee_A but not in Employee_B.

$$Employee_A - Employee_B$$

5. Cartesian Product ($\times$)

Combine every employee with every department.

$$Employee \times Department$$

6. Join (⋈)

Retrieve employee names with their department names.

$Employee \bowtie_{Employee.DeptID=Department.DeptID} Department$

Result

The above relational algebra expressions demonstrate the required operations:

- σ (**Selection**) – Filters rows.
- π (**Projection**) – Selects columns.
- $\cup$ (**Union**) – Combines tuples.
- $-$ (**Difference**) – Finds unmatched tuples.
- $\times$ (**Cartesian Product**) – Produces all possible combinations.
- $\bowtie$ (**Join**) – Combines related records from Employee and Department tables.

Q2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

Ans:

-- Create Department Table

```
CREATE TABLE DEPARTMENT (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) NOT NULL  
);
```

-- Create Student Table

```
CREATE TABLE STUDENT (  
    SID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    DOB DATE NOT NULL,  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)  
);
```

Constraints Used

- PRIMARY KEY → DeptID, SID (Uniquely identifies each record)
- NOT NULL → DeptName, Name, DOB (Cannot contain NULL values)
- FOREIGN KEY → DeptID in STUDENT references DEPARTMENT(DeptID) to maintain referential integrity.

Q3: Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

Assume Table: SALES

SALES(SaleID, ProductName, Category, Quantity, Amount)

1. COUNT() with GROUP BY

```
SELECT Category, COUNT(*) AS TotalSales
FROM SALES
GROUP BY Category;
```

2. SUM() with GROUP BY

```
SELECT Category, SUM(Amount) AS TotalAmount
FROM SALES
GROUP BY Category;
```

3. AVG() with GROUP BY

```
SELECT Category, AVG(Amount) AS AverageAmount
FROM SALES
GROUP BY Category;
```

4. MAX() and MIN() with GROUP BY

```
SELECT Category,
MAX(Amount) AS MaximumAmount,
MIN(Amount) AS MinimumAmount
FROM SALES
GROUP BY Category;
```

5. HAVING Clause

Display categories whose total sales amount is greater than 50000.

```
SELECT Category, SUM(Amount) AS TotalAmount
FROM SALES
GROUP BY Category
HAVING SUM(Amount) > 50000;
```

Q4: Write a correlated sub-query to find employees earning more than the average salary of their own department.

Answer

Assume Table:

EMPLOYEE(EmpID, EmpName, Salary, DeptID)

Correlated Sub-Query

```
SELECT EmpID, EmpName, Salary, DeptID
FROM EMPLOYEE E
WHERE Salary >
(
    SELECT AVG(Salary)
    FROM EMPLOYEE
    WHERE DeptID = E.DeptID
);
```

Explanation

- The **outer query** selects employee details.
- The **inner (correlated) query** calculates the **average salary** of the employee's own department.
- The WHERE DeptID = E.DeptID condition links the inner query with the outer query.
- The query displays employees whose **salary is greater than the average salary of their department**.

Q5: Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

Assume Tables

EMPLOYEE(EmpID, EmpName, ProjectID)

PROJECT(ProjectID, ProjectName)

1. INNER JOIN

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
INNER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

2. LEFT OUTER JOIN

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
LEFT OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

3. RIGHT OUTER JOIN

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
RIGHT OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

4. FULL OUTER JOIN

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
FULL OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

Note: MySQL does not support FULL OUTER JOIN directly. It can be achieved using the UNION of LEFT JOIN and RIGHT JOIN.

5. CROSS JOIN

```
SELECT E.EmpID, E.EmpName, P.ProjectName
```

FROM EMPLOYEE E
CROSS JOIN PROJECT P;

Result

- **INNER JOIN** → Returns only matching records.
- **LEFT OUTER JOIN** → Returns all employees and matching projects.
- **RIGHT OUTER JOIN** → Returns all projects and matching employees.
- **FULL OUTER JOIN** → Returns all records from both tables.
- **CROSS JOIN** → Returns every possible combination of employees and projects.

Q6: Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.

Answer

Create View

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
       COUNT(E.EmpID) AS TotalEmployees,
       AVG(E.Salary) AS AverageSalary
FROM EMPLOYEE E
JOIN DEPARTMENT D
ON E.DeptID = D.DeptID
GROUP BY D.DeptName;
```

Query on the View

```
SELECT * FROM DeptSalaryView;
```

Explanation

- **CREATE VIEW** creates a virtual table named **DeptSalaryView**.
- **COUNT(E.EmpID)** calculates the total number of employees in each department.
- **AVG(E.Salary)** calculates the average salary of employees in each department.
- **GROUP BY D.DeptName** groups the records department-wise.
- **SELECT * FROM DeptSalaryView** displays the department name, total employees, and average salary.

Q7: Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

Answer

Assume Relations:

STUDENT(SID, Name)

COURSE(CID, CourseName)

ENROLL(SID, CID)

Relational Algebra Expression

```
 $\pi_{\text{Name}} (\text{STUDENT} \bowtie ( \pi_{\text{SID,CID}}(\text{ENROLL}) \div \pi_{\text{CID}}(\text{COURSE}) ) )$ 
```

Explanation

- $\pi_{CID}(COURSE) \rightarrow$ Retrieves all Course IDs offered.
- $\pi_{SID,CID}(ENROLL) \rightarrow$ Retrieves Student IDs with the Course IDs they are enrolled in.
- $\div$ (**Division Operator**) $\rightarrow$ Finds students who are enrolled in **all** courses offered.
- $\bowtie$ (**Join**) $\rightarrow$ Joins the result with the **STUDENT** relation.
- $\pi_{Name} \rightarrow$ Displays only the names of those students.

Q8: Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

Answer

Assume Table:

STUDENT(SID, Name, DOB, DeptID)

1. INSERT Statement

```
INSERT INTO STUDENT (SID, Name, DOB, DeptID)
VALUES (101, 'Rahul', '2005-08-15', 10);
```

2. UPDATE Statement

```
UPDATE STUDENT
SET DeptID = 20
WHERE SID = 101;
```

3. DELETE Statement

```
DELETE FROM STUDENT
WHERE SID = 101;
```

Integrity Constraint Enforcement

- **PRIMARY KEY** $\rightarrow$ SID must be unique.
- **NOT NULL** $\rightarrow$ Name and DOB cannot be NULL.
- **FOREIGN KEY** $\rightarrow$ DeptID must exist in the DEPARTMENT table before inserting or updating.
- If these constraints are violated, the database rejects the operation and maintains data integrity.

Q9: Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

Answer

Assume Relation:

EMPLOYEE(EmpID, EmpName, Salary, DeptName)

Tuple Relational Calculus (TRC)

{ E | EMPLOYEE(E)

$\wedge E.DeptName = 'IT'$

$\wedge E.Salary > 50000 \}$

Explanation

- **E** → Tuple variable representing each employee.
- **EMPLOYEE(E)** → Specifies that E belongs to the EMPLOYEE relation.
- **E.DeptName = 'IT'** → Selects employees working in the **IT** department.
- **E.Salary > 50000** → Selects employees whose salary is greater than **50,000**.
- The query returns **all employees who work in the IT department and earn more than ₹50,000**.

Q10: Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

Answer

Relational Schema

PASSENGER(PassengerID, PassengerName, Age, Phone)

FLIGHT(FlightID, FlightName, Source, Destination)

BOOKING(BookingID, PassengerID, FlightID, JourneyDate)

PAYMENT(PaymentID, BookingID, Amount, PaymentMode)

1. JOIN Query

```
SELECT P.PassengerName, F.FlightName
FROM PASSENGER P
JOIN BOOKING B
ON P.PassengerID = B.PassengerID
JOIN FLIGHT F
ON B.FlightID = F.FlightID;
```

2. Sub-Query

Display passengers who booked Flight ID = 101.

```
SELECT PassengerName
FROM PASSENGER
WHERE PassengerID IN
(
    SELECT PassengerID
    FROM BOOKING
    WHERE FlightID = 101
);
```

3. Aggregate Query

```
SELECT FlightID, COUNT(*) AS TotalBookings
FROM BOOKING
GROUP BY FlightID;
```

4. Create View

```
CREATE VIEW FlightBookingView AS
SELECT P.PassengerName,
       F.FlightName,
       B.JourneyDate
FROM PASSENGER P
JOIN BOOKING B
ON P.PassengerID = B.PassengerID
JOIN FLIGHT F
ON B.FlightID = F.FlightID;
```

5. Query on View

```
SELECT * FROM FlightBookingView;
```